

UAV MOTION PLANNING HANDOUT

Planning as a Markov Decision Process

A deep mathematical treatment from Bellman equations to safe UAV policies

Bijay Gaudel

Central idea. A planner chooses actions so that the distribution of future states produces low cost, high reward, and acceptable risk. An MDP makes every part of that sentence explicit: state, action, transition law, objective, information pattern, and policy.

1 The controlled stochastic process

A discounted Markov decision process is the tuple

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0),$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $P(s' | s, a)$ a transition kernel, $r(s, a, s')$ an immediate reward, $\gamma \in [0, 1)$ a discount factor, and ρ_0 an initial-state distribution. In a finite MDP,

$$P(s' | s, a) \geq 0, \quad \sum_{s' \in \mathcal{S}} P(s' | s, a) = 1.$$

The Markov property is

$$\mathbb{P}(S_{t+1} = s' | S_0, A_0, \dots, S_t, A_t) = P(s' | S_t, A_t).$$

It does not claim the physical world has no memory. It claims the chosen state contains enough information that the remaining history adds no predictive power for the next-state distribution.

For a UAV, a state might be

$$s_t = (p_t, v_t, R_t, \omega_t, b_t, m_t),$$

containing position, velocity, attitude, angular velocity, battery state, and a map or belief representation. If wind gust dynamics are temporally correlated but omitted, position alone is not Markov.

2 Policies, trajectory distributions, and return

A stochastic Markov policy is $\pi(a | s)$, with $\sum_a \pi(a | s) = 1$. A deterministic policy is a map $\pi : \mathcal{S} \rightarrow \mathcal{A}$. Given ρ_0 , P , and π , the probability of a horizon- T trajectory

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$$

factorizes as

$$p_\pi(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi(a_t | s_t) P(s_{t+1} | s_t, a_t).$$

The discounted return from time t is

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

The planning objective is

$$J(\pi) = \mathbb{E}_{\tau \sim p_\pi}[G_0].$$

Discounting ensures bounded returns when rewards are bounded, values near-term outcomes more strongly, and can be interpreted as geometric survival with continuation probability γ . For episodic planning one may instead use a finite horizon or a stochastic-shortest-path formulation with $\gamma = 1$ and absorbing terminal states.

3 Value functions and the Bellman expectation equations

For a fixed policy π ,

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s], \quad Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Split the return into its first reward and the remaining return:

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

Conditioning on the next state and action gives

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) [r(s, a, s') + \gamma V^\pi(s')],$$

$$Q^\pi(s, a) = \sum_{s'} P(s' \mid s, a) \left[r(s, a, s') + \gamma \sum_{a'} \pi(a' \mid s') Q^\pi(s', a') \right].$$

These are identities, not approximations. They express temporal consistency: the value assigned now must agree with one expected transition plus the value assigned afterward.

Define the policy reward vector and transition matrix

$$r_\pi(s) = \sum_{a, s'} \pi(a \mid s) P(s' \mid s, a) r(s, a, s'), \quad P_\pi(s, s') = \sum_a \pi(a \mid s) P(s' \mid s, a).$$

Then

$$V^\pi = r_\pi + \gamma P_\pi V^\pi, \quad V^\pi = (I - \gamma P_\pi)^{-1} r_\pi.$$

The inverse exists for $\gamma < 1$ because the spectral radius of γP_π is below one.

4 Bellman optimality

Define

$$V^*(s) = \max_\pi V^\pi(s), \quad Q^*(s, a) = \max_\pi Q^\pi(s, a).$$

After the first action, optimal behavior may be used from the next state onward. This principle of optimality yields

$$V^*(s) = \max_a \sum_{s'} P(s' \mid s, a) [r(s, a, s') + \gamma V^*(s')],$$

$$Q^*(s, a) = \sum_{s'} P(s' \mid s, a) \left[r(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right].$$

An optimal deterministic stationary policy exists for every finite discounted MDP and can be selected greedily:

$$\pi^*(s) \in \arg \max_a Q^*(s, a).$$

5 Bellman operators and contraction

Define the policy and optimality operators

$$(\mathcal{T}^\pi V)(s) = \sum_{a, s'} \pi(a \mid s) P(s' \mid s, a) [r(s, a, s') + \gamma V(s')],$$

$$(\mathcal{T}V)(s) = \max_a \sum_{s'} P(s' \mid s, a) [r(s, a, s') + \gamma V(s')].$$

Theorem 1 (Contraction). *For any bounded value functions V, W ,*

$$\|\mathcal{T}V - \mathcal{T}W\|_\infty \leq \gamma \|V - W\|_\infty.$$

The same holds for $\mathcal{T}^\pi$.

Proof. For numbers x_a, y_a , $|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|$. Therefore, for each s ,

$$\begin{aligned} |(\mathcal{TV})(s) - (\mathcal{TW})(s)| &\leq \max_a \gamma \left| \sum_{s'} P(s' | s, a) (V(s') - W(s')) \right| \\ &\leq \gamma \max_a \sum_{s'} P(s' | s, a) \|V - W\|_\infty \\ &= \gamma \|V - W\|_\infty. \end{aligned}$$

Taking the maximum over s proves the claim. $\square$

By the Banach fixed-point theorem, $\mathcal{T}$ has a unique fixed point V^* , and repeated application converges from any bounded initialization:

$$V_{k+1} = \mathcal{TV}_k, \quad \|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty.$$

If $|r| \leq R_{\max}$ and $V_0 = 0$, then

$$\|V_k - V^*\|_\infty \leq \frac{\gamma^k R_{\max}}{1 - \gamma}.$$

6 Value iteration

Algorithm 1 Value iteration for a finite discounted MDP

Require: P, r, γ , tolerance ε

- 1: Initialize $V(s)$ arbitrarily
 - 2: **repeat**
 - 3: $\Delta \leftarrow 0$
 - 4: **for all** $s \in \mathcal{S}$ **do**
 - 5: $v \leftarrow V(s)$
 - 6: $V(s) \leftarrow \max_a \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma V_{\text{old}}(s')]$
 - 7: $\Delta \leftarrow \max(\Delta, |V(s) - v|)$
 - 8: **until** Δ is below the selected residual threshold
 - 9: $\pi(s) \leftarrow \arg \max_a \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma V(s')]$
-

Synchronous updates use an unchanged copy V_{old} . Gauss–Seidel updates reuse new components immediately and often converge faster. Prioritized sweeping focuses backups where the Bellman residual

$$\delta_V(s) = |(\mathcal{TV})(s) - V(s)|$$

is large.

The residual gives an error certificate. Since

$$\|V - V^*\|_\infty \leq \frac{\|\mathcal{TV} - V\|_\infty}{1 - \gamma},$$

one can stop based on a desired value error, not merely an arbitrary iteration count.

7 Policy iteration and policy improvement

Policy iteration alternates:

1. **Evaluation:** solve $V^{\pi_k} = \mathcal{T}^{\pi_k} V^{\pi_k}$, exactly or iteratively;
2. **Improvement:**

$$\pi_{k+1}(s) \in \arg \max_a \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma V^{\pi_k}(s')].$$

Theorem 2 (Policy improvement). *If π' is greedy with respect to V^π , then $V^{\pi'}(s) \geq V^\pi(s)$ for all s .*

Proof. Greediness implies

$$\mathcal{T}^{\pi'} V^\pi = \mathcal{T} V^\pi \geq \mathcal{T}^\pi V^\pi = V^\pi.$$

Monotonicity of $\mathcal{T}^{\pi'}$ gives

$$V^\pi \leq \mathcal{T}^{\pi'} V^\pi \leq (\mathcal{T}^{\pi'})^2 V^\pi \leq \dots \rightarrow V^{\pi'}.$$

□

In a finite MDP, strict improvements cannot continue forever because there are finitely many deterministic policies. Modified policy iteration performs only a few evaluation sweeps before improving, interpolating between value and policy iteration.

8 Finite-horizon dynamic programming

For a horizon T with terminal value $V_T(s)$, optimal policies may depend on time. Backward induction gives

$$V_t(s) = \max_a \sum_{s'} P(s' | s, a) [r_t(s, a, s') + V_{t+1}(s')], \quad t = T-1, \dots, 0.$$

There is no fixed-point iteration: the terminal condition anchors a backward recursion. This form naturally models battery-limited missions, deadlines, or a receding planning horizon. A stationary policy can be suboptimal when the number of remaining steps matters; augmenting state with time restores stationarity in the augmented MDP.

9 Shortest paths as deterministic MDPs

Let each graph vertex be a state, each outgoing edge an action, and transitions be deterministic:

$$P(s' | s, a) = \mathbf{1}\{s' = F(s, a)\}.$$

For edge cost $c(s, a) \geq 0$, use reward $r = -c$, an absorbing goal, and an undiscounted stochastic-shortest-path objective. In cost form,

$$J^*(s) = \min_a [c(s, a) + J^*(F(s, a))], \quad J^*(s_g) = 0.$$

This is a Bellman equation. Dijkstra exploits nonnegative deterministic costs to order backups so each finalized label is exact. A* adds a lower bound on cost-to-go to focus those backups.

The equivalence must be stated carefully: Dijkstra is not identical to ordinary synchronous value iteration on an arbitrary discounted stochastic MDP. It is a specialized label-setting dynamic-programming method for deterministic shortest paths with nonnegative edge costs.

10 Stochastic shortest-path MDPs

An SSP has an absorbing goal s_g , nonnegative stage cost $c(s, a)$, and typically $\gamma = 1$. The objective is expected total cost until absorption:

$$J^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\tau_g-1} c(S_t, A_t) \mid S_0 = s \right].$$

A policy is *proper* if it reaches the goal with probability one from every relevant initial state. Unlike the discounted case, the Bellman operator is not generally a contraction in the ordinary sup norm. Standard SSP convergence results require conditions such as existence of a proper policy and infinite expected cost for improper policies. Zero-cost cycles require particular care.

For UAV planning under wind, an intended move may produce several successor cells. The SSP Bellman equation is

$$J^*(s) = \min_a \left[c(s, a) + \sum_{s'} P(s' | s, a) J^*(s') \right].$$

The policy may deliberately choose a longer geometric route if it has lower expected delay or collision exposure.

11 A worked stochastic UAV cell

At state s , suppose action DIRECT has immediate cost 1 and reaches the goal with probability 0.8, but wind sends the vehicle to recovery state b with probability 0.2. Action SAFE costs 2 and reaches the goal with certainty. Let $J(b) = 8$ and $J(s_g) = 0$. Then

$$Q(s, \text{DIRECT}) = 1 + 0.8(0) + 0.2(8) = 2.6,$$

$$Q(s, \text{SAFE}) = 2 + 1(0) = 2.$$

The geometrically shorter action is worse in expected cost. If $J(b)$ were only 3, Direct would cost 1.6 and become optimal. MDP planning makes this trade explicit rather than treating disturbances as a post-planning correction.

12 Linear programming and occupancy measures

For a finite discounted reward-maximization MDP, V^* is the solution of

$$\min_V \sum_s \mu(s) V(s)$$

subject to the Bellman inequalities

$$V(s) \geq \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma V(s')] \quad \forall (s, a),$$

where $\mu(s) > 0$. The smallest feasible upper bound is the Bellman fixed point.

The dual uses discounted state-action occupancy

$$d_\pi(s, a) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}_\pi(S_t = s, A_t = a).$$

It satisfies flow conservation:

$$\sum_a d(s, a) = (1 - \gamma) \rho_0(s) + \gamma \sum_{\bar{s}, \bar{a}} d(\bar{s}, \bar{a}) P(s | \bar{s}, \bar{a}).$$

The objective is linear in occupancy. A policy is recovered by

$$\pi(a | s) = \frac{d(s, a)}{\sum_{a'} d(s, a')}$$

when the denominator is nonzero. This viewpoint is powerful for constraints because expected cumulative resources are linear in d .

13 Constrained and risk-sensitive planning

A constrained MDP maximizes reward while limiting expected costs:

$$\max_{\pi} J_r(\pi) \quad \text{s.t.} \quad J_{c_i}(\pi) \leq b_i, \text{quad} i = 1, \dots, m.$$

Examples include expected energy, no-fly-zone exposure, thermal load, or communication outage. The Lagrangian

$$L(\pi, \lambda) = J_r(\pi) - \sum_i \lambda_i (J_{c_i}(\pi) - b_i), \quad \lambda_i \geq 0,$$

converts constraints into adaptive penalties, but satisfying the expectation constraint does not imply per-flight safety.

Risk-sensitive criteria include variance penalties, exponential utility, and conditional value at risk (CVaR). A chance constraint

$$\mathbb{P}_\pi(\text{collision during mission}) \leq \alpha$$

is different from adding a large collision penalty: a penalty trades collision probability against reward, whereas a constraint declares an acceptable risk budget. Safety-critical deployment generally needs robust or constrained design, verification, and a fallback controller beyond a nominal expected-return policy.

14 Reward design and policy invariance

Reward terms must represent the desired mission, not a convenient proxy. Common failures include orbiting near a goal to harvest shaping reward, moving slowly to avoid energy penalties forever, or exploiting simulator termination details.

Potential-based shaping adds

$$F(s, a, s') = \gamma\Phi(s') - \Phi(s)$$

to the original reward. The shaped return telescopes, changing values by a state-dependent constant while preserving optimal policies under standard discounted assumptions. Arbitrary dense shaping does not have this guarantee.

For shortest-path-style planning, a clean construction is a negative step cost, terminal success reward or zero terminal cost, explicit collision termination, and a bounded potential based on progress. Every scale and unit should be documented.

15 Known-model planning versus reinforcement learning

If P and r are known, Bellman backups can sum over successors: this is model-based planning. If the model is unknown but transitions (s, a, r, s') can be sampled, reinforcement learning estimates values or policies from data.

Q-learning uses the stochastic approximation

$$Q_{t+1}(S_t, A_t) = Q_t(S_t, A_t) + \alpha_t \left[R_{t+1} + \gamma \max_{a'} Q_t(S_{t+1}, a') - Q_t(S_t, A_t) \right].$$

The bracketed term is the temporal-difference error. In a finite tabular MDP, Q-learning converges to Q^* with probability one under conditions including bounded rewards, sufficient visitation of every state-action pair, and step sizes satisfying

$$\sum_t \alpha_t = \infty, \quad \sum_t \alpha_t^2 < \infty.$$

These conditions do not automatically hold with function approximation, partial observability, changing dynamics, or unsafe exploration. Planning and learning solve related Bellman equations but have very different information and guarantee assumptions.

16 Partial observability and belief-state planning

If the agent observes o_t rather than the true state, the problem is a POMDP. The sufficient statistic is the belief

$$b_t(s) = \mathbb{P}(S_t = s \mid o_{0:t}, a_{0:t-1}).$$

After action a and observation o' , Bayes filtering gives

$$b'(s') = \eta O(o' \mid s', a) \sum_s P(s' \mid s, a) b(s),$$

with normalizer η . The belief is a continuous state even when the physical state is finite. Planning can then be written as an MDP over beliefs, but exact solution is computationally difficult. For UAVs, map uncertainty, occlusion, localization ambiguity, and hidden wind modes can make belief-state reasoning essential.

17 Continuous states, actions, and approximation

For continuous spaces, sums become integrals:

$$V^*(s) = \sup_a \int P(ds' \mid s, a) [r(s, a, s') + \gamma V^*(s')].$$

Exact tabulation is impossible. Options include discretization, fitted value iteration, approximate dynamic programming, model-predictive control, sampling-based tree search, and actor-critic methods. Approximation creates three separate errors:

- **model error:** $\hat{P}$ and $\hat{r}$ differ from the real system;

- **representation error:** the chosen function class cannot express V^* or π^* ;
- **optimization/statistical error:** finite computation and finite data do not find the best representable solution.

These should not be collapsed into a single “RL error.”

18 Building an MDP for UAV motion planning

1. **Choose a decision timescale.** Actions might be motor commands at 100 Hz, acceleration commands at 10 Hz, or motion primitives every second. The choice changes both Markov state and computational burden.
2. **Define state sufficiency.** Include velocity for inertial motion, battery for energy-dependent feasibility, and a wind estimate or belief if disturbances are correlated.
3. **Define feasible actions.** Encode actuator, attitude, thrust, speed, and airspace limits before optimization when possible.
4. **Specify transitions.** Use a dynamics model plus disturbance distribution; validate calibration and out-of-distribution behavior.
5. **Specify termination.** Separate success, collision, timeout, battery exhaustion, and leaving the operational region.
6. **Choose objective and risk semantics.** Distinguish expected performance from worst-case or chance-constrained safety.
7. **Select a solver whose assumptions match.** Finite known model suggests DP; deterministic sparse graph suggests Dijkstra/A*; continuous online replanning suggests MPC; unknown dynamics may motivate learning.
8. **Verify empirically and analytically.** Report return distributions, success probability, collision probability, constraint violations, model mismatch, and sensitivity—not mean reward alone.

19 Common mathematical and implementation mistakes

- Calling a process Markov while omitting velocity, time, battery, map belief, or disturbance memory.
- Mixing reward maximization and cost minimization signs inside one Bellman equation.
- Using $\gamma < 1$ in a task where discounting unintentionally rewards delaying a negative terminal event.
- Treating a high collision penalty as a proof of safety.
- Claiming Dijkstra or A* solves an arbitrary stochastic discounted MDP.
- Performing in-place value iteration while describing it as synchronous without documenting update order.
- Stopping from raw iterate difference without converting the Bellman residual into an error bound.
- Evaluating a learned policy only in the same simulator distribution used for training.
- Reporting average return without failure probability, tail risk, or confidence intervals.
- Assuming model-free means assumption-free: state, reward, stationarity, coverage, and function approximation remain modeling choices.

20 Summary map

Setting	Core equation or object	Typical method
Finite discounted MDP	$V^* = \mathcal{T}V^*, \gamma < 1$	value iteration, policy iteration, LP
Finite horizon	backward Bellman recursion	backward induction, receding horizon
Deterministic shortest path	$J^*(s) = \min_a [c + J^*(s')]$	Dijkstra, A^*
Stochastic shortest path	expected cost to absorption	SSP value/policy iteration
Unknown finite MDP	sampled Bellman targets	Q-learning, model learning
Partial observation	belief-state MDP	filtering plus POMDP planning
Continuous spaces	integral Bellman equation	approximation, MPC, sampling, RL
Safety/resource limits	occupancy and constraints	CMDP, robust/risk-sensitive methods

The MDP framework is not one algorithm. It is the mathematical language that states what information is available, how actions alter uncertainty, what objective is optimized, and which guarantees are meaningful. A correct planner begins by making those assumptions explicit.